

UNIVERSIDAD DE LAS FUERZAS ARMADAS – ESPE

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

CARRERA DE INGENIERÍA EN SOFTWARE

INFORME DE LABORATORIO

PROYECTO 8 – ALTERNATIVA 5

TIC-TAC-TOE EN .NET MAUI

INTELIGENCIA ARTIFICIAL CON MINIMAX

ASIGNATURA: DESARROLLO DE VIDEOJUEGOS

UNIDAD 1

DOCENTE: ING. CÉSAR VILLACÍS SILVA, MSC.

ESTUDIANTES:

_____

_____

_____

GRUPO N.º: _____

FECHA: NOVIEMBRE 2026

Resumen

Este informe documenta una aplicación de tres en raya para Android, implementada en C# con .NET MAUI. La persona juega con `X`; la computadora responde con `O` mediante una búsqueda Minimax que recorre las jugadas legales restantes, evalúa victorias, derrotas y empates y favorece ganar pronto o retrasar una derrota. El documento sigue la guía de laboratorio proporcionada para la asignatura: especifica requisitos funcionales y no funcionales, presenta diagramas de casos de uso, secuencia y clases, explica el diseño y describe el trabajo con una adaptación de SRUM+XP.

La revisión se apoya en el código del proyecto MAUI, en la configuración del proyecto, en tres capturas de Visual Studio Community 2026 y en las evidencias existentes del emulador. Estas últimas incluyen 14 imágenes y 25 verificaciones registradas. Los resultados documentan el turno de la IA, el bloqueo de una amenaza, una victoria de la computadora, un empate, el reinicio y la salida. También dejan visible que las opciones de los menús Archivo, Ver y Ayuda aparecen en pantalla pero no ejecutan las acciones anunciadas. El documento separa esos hallazgos de las mejoras propuestas y deja campos editables para los nombres y el grupo de trabajo.

Palabras clave: videojuegos, .NET MAUI, Android, C#, Minimax, Tic-Tac-Toe, SRUM, XP.

Índice general

Resumen **i**

1 Introducción y alcance **1**

1.1 Contexto 1

1.2 Objetivos 1

1.2.1 Objetivo general 1

1.2.2 Objetivos específicos 1

1.3 Alcance del producto 2

1.4 Términos y abreviaturas 2

2 Especificación de requisitos **3**

2.1 Convenciones y base de la especificación 3

2.2 Requisitos funcionales 4

2.3 Requisitos no funcionales 5

2.4 Criterios de aceptación 6

3 Diseño del sistema **7**

3.1 Arquitectura 7

3.2 Representación del tablero 7

3.3 Diagrama de casos de uso 8

3.3.1 Descripción resumida de casos de uso 9

3.4 Diagrama de secuencia de una jugada 9

3.5 Diagrama de clases 9

4 Algoritmo Minimax **11**

4.1 Idea de decisión 11

4.2 Procedimiento implementado 12

4.3 Condiciones terminales y desempate 12

4.4 Costo de búsqueda 13

5 Desarrollo con SRUM+XP **14**

5.1 Adaptación de la metodología 14
5.2 Prácticas XP observables 15
5.3 Backlog de mejoras priorizado 15

6 Implementación y configuración en Visual Studio 16
6.1 Estructura del proyecto 16
6.2 Configuración comprobada 16
6.3 Capturas de configuración en Visual Studio 17
6.4 Pasos para abrir y ejecutar desde Visual Studio 17
6.5 Configuración Android que conviene conservar 19

7 Validación y métricas 20
7.1 Evidencia de ejecución existente 20
7.2 Métricas estáticas 21
7.3 Observaciones sobre la calidad de la evidencia 21

8 Análisis, limitaciones y conclusiones 22
8.1 Análisis 22
8.2 Limitaciones detectadas 22
8.3 Conclusiones 22
8.4 Recomendación de entrega 23

A Evidencias de la aplicación en el emulador 25

Índice de figuras

3.1 Arquitectura lógica de la aplicación MAUI 7
3.2 Casos de uso de la aplicación 8
3.3 Secuencia de una jugada y búsqueda Minimax 10
3.4 Clases centrales de la lógica del juego 10

6.1 Visual Studio Community 2026 con el proyecto cargado. La barra muestra Debug, Any CPU y el emulador Android seleccionado; el editor muestra el destino `net10.0-android`. 18
6.2 Configuración de aplicación y plataforma en el `.csproj`: título, Application ID, versiones y mínimo Android API 21. 18
6.3 Recursos MAUI y referencias de paquetes declarados en el proyecto. 19

A.1 Pantalla inicial con el tablero vacío, los símbolos de cada participante y los controles visibles. 26
A.2 La persona coloca `X` y Minimax responde con `O`. 27
A.3 Verificación registrada para la casilla ocupada; la imagen conserva el mismo hash que la captura 02. 28
A.4 Hoja de acciones del menú Archivo. Las opciones no están conectadas a manejadores de acción. 29
A.5 Hoja de acciones del menú Ver. Puntajes y Resetear puntajes son etiquetas visibles, sin función implementada. 30
A.6 Hoja de acciones del menú Ayuda; las selecciones no abren contenido. 31
A.7 Minimax bloquea la amenaza de victoria de `X` en la primera columna. 32
A.8 Aviso de victoria de la IA registrado en el escenario de prueba. 33
A.9 Tablero con victoria diagonal de `O`; la entrada posterior queda bloqueada. 34
A.10 El botón Nuevo limpia la partida y habilita nuevamente las casillas. 35
A.11 Aviso de empate al completar el tablero sin una línea ganadora. 36
A.12 Estado final del tablero empatado. 37
A.13 Pantalla de inicio de Android tras usar el botón Salir. 38
A.14 Aplicación reabierta con una partida nueva y tablero vacío. 39

Índice de tablas

1.1 Términos y abreviaturas 2

2.1 Requisitos funcionales y evidencia asociada 4
2.2 Requisitos no funcionales y observaciones 5

3.1 Relación entre índice lineal y coordenadas del tablero 8
3.2 Casos de uso y condición observada 9

5.1 Incrementos reconstruidos a partir del proyecto 14
5.2 Relación entre prácticas ágiles y evidencia encontrada 15

6.1 Archivos de entrada más relevantes 16
6.2 Propiedades del proyecto y valores encontrados 17

7.1 Resumen de escenarios de aceptación documentados 20
7.2 Métricas observables del proyecto 21

Capítulo 1

Introducción y alcance

1.1 Contexto

La guía de la asignatura propone implementar Tic-Tac-Toe con una inteligencia artificial MinMax y C# MAUI para móvil, dentro del Proyecto 8, alternativa 5. También solicita carátula, requisitos, diagramas, aplicación de SRUM+XP y métricas de software [1]. Este trabajo documenta la aplicación ubicada en `Tic-Tac-Toe/MinMax/C#_MAUI_Mobile`.

El sistema presenta un tablero de 3×3. La persona juega primero y utiliza `X`; el programa utiliza `O`. La interfaz se construye con XAML y controles de .NET MAUI. La página maneja el estado del tablero; un controlador independiente calcula la respuesta de la computadora.

Hay una diferencia entre dos materiales de referencia del directorio. La guía DOCX define el formato académico y los entregables. El DOCX denominado *minmax-ia-explicacion* explica una versión Windows Forms, con una clase `frmTicTacToe`; esa clase no pertenece al proyecto MAUI revisado. Por ello, la explicación de la interfaz de este informe se basa en `MainPage` y el algoritmo se describe según el controlador Minimax que sí está en el proyecto.

1.2 Objetivos

1.2.1 Objetivo general

Describir el diseño, la implementación y la evidencia disponible de un juego móvil de tres en raya en .NET MAUI cuya computadora selecciona movimientos mediante Minimax.

1.2.2 Objetivos específicos

1. Identificar el alcance funcional y las restricciones observables en el código.
2. Explicar cómo se representa el tablero y cómo se elige una jugada óptima.
3. Documentar la configuración Android y los pasos para abrir y ejecutar el proyecto desde Visual Studio.
4. Relacionar las capacidades implementadas con los resultados y límites de la evidencia existente.
5. Registrar métricas estáticas y proponer trabajo pendiente sin atribuir al proyecto funciones que no implementa.

1.3 Alcance del producto

El alcance implementado es una aplicación Android de una sola pantalla, con tablero, turnos, avisos de fin de partida, botón Nuevo y botón Salir. La plataforma objetivo declarada es `net10.0-android`; que el proyecto use el modelo de proyecto único de MAUI no significa que el archivo de proyecto actual esté configurado para publicar en iOS, Mac Catalyst o Windows. .NET MAUI permite un proyecto compartido para varias plataformas, pero cada una debe estar incluida como destino de compilación [3].

Quedan fuera del comportamiento implementado el almacenamiento de puntuaciones, la selección de dificultad, la persistencia de partidas, el contenido de ayuda y la ejecución de las opciones internas de los menús. Las etiquetas de esas opciones sí aparecen, pero el código no procesa la selección que devuelve cada hoja de acciones.

1.4 Términos y abreviaturas

Tabla 1.1 Términos y abreviaturas

Término	Definición usada en este informe
Estado	Distribución de `X`, `O` y casillas vacías en la matriz de 3×3.
Nodo	Estado del tablero considerado durante la búsqueda del árbol de jugadas.
Jugador MAX	La IA, representada por `O`, que maximiza la puntuación.
Jugador MIN	La persona, representada por `X`, que minimiza la puntuación de la IA.
MAUI	Interfaz de usuario multiplataforma de .NET; el proyecto revisado compila para Android.
SRUM+XP	Adaptación solicitada por la guía académica para organizar diseño, desarrollo y pruebas.

Capítulo 2

Especificación de requisitos

2.1 Convenciones y base de la especificación

La guía del curso solicita requisitos según IEEE 830 [1]. IEEE SA registra IEEE 830-1998 como sustituido por ISO/IEC/IEEE 29148:2011. La edición 2018 de 29148 figura como estándar activo [5, 6]. Este informe conserva el formato de especificación de la asignatura y no afirma una certificación de conformidad.

Los estados de cumplimiento se interpretan así: Implementado indica que el código contiene el comportamiento; Parcial indica que se cumple una parte; Pendiente indica que el comportamiento solicitado no se encontró en el código revisado. Una captura de pantalla prueba un escenario visible, no todas las entradas posibles.

2.2 Requisitos funcionales

Tabla 2.1 Requisitos funcionales y evidencia asociada

ID	Requisito	Estado	Evidencia
RF-01	Al iniciar, mostrar nueve casillas vacías y permitir que la persona empiece con `X`.	Implementado	`ResetBoard`; captura 01.
RF-02	Registrar una jugada humana solo si la casilla está vacía y es el turno humano.	Implementado	`OnCellClicked`; captura 03.
RF-03	Después de una jugada humana válida, calcular y registrar una respuesta de la IA con `O`.	Implementado	`AIMoveAsync`, `GetBestMove`; captura 02.
RF-04	Detectar victoria de `X`, victoria de `O` o empate; mostrar aviso y bloquear jugadas posteriores.	Implementado	`CheckGameEnd Async`; capturas 08, 09 y 11.
RF-05	Reiniciar la matriz, habilitar las nueve casillas y devolver el turno a la persona.	Implementado	`OnNewClicked`; captura 10.
RF-06	Cerrar la aplicación desde el control Salir.	Implementado	`OnExitClicked`; capturas 13 y 14.
RF-07	Mostrar las opciones visibles Archivo, Ver y Ayuda.	Implementado	`MainPage.xaml`; capturas 04, 05 y 06.
RF-08	Ejecutar la acción elegida en cada menú: Nuevo, Salir, Puntajes, Resetear puntajes, Acerca de y Contenido.	Pendiente	Los manejadores solo muestran la hoja de acciones y descartan su selección.

2.3 Requisitos no funcionales

Tabla 2.2 Requisitos no funcionales y observaciones

ID	Requisito	Estado	Nota / límite
RNF-01	La aplicación debe orientarse a Android y declarar una versión mínima de plataforma.	Declarado	`net10.0-android`; mínimo Android 5.0/API 21 en el proyecto.
RNF-02	El tablero debe conservar proporción cuadrada y adaptarse al ancho disponible.	Implementado	`OnSizeAllocated`; ancho máximo de 340 unidades MAUI.
RNF-03	Los controles deben ser identificables por automatización y contar con descripción semántica.	Parcialmente implementado	Cada casilla recibe `AutomationId` y descripción; no hay evidencia de evaluación formal con lector de pantalla.
RNF-04	La jugada de la IA debe responder con rapidez suficiente para un tablero 3×3.	No medido	La búsqueda se ejecuta sincrónicamente y no se guardan tiempos de respuesta.
RNF-05	La lógica del algoritmo debe poder leerse separada de la interfaz.	Implementado	El controlador Minimax es una clase aparte de `MainPage`.
RNF-06	No se debe aceptar una jugada en una casilla ocupada ni continuar una partida terminada.	Implementado	Validaciones en `OnCellClicked`; evidencia de ocupación y fin de partida.
RNF-07	La aplicación debe minimizar permisos que no necesita.	Revisar	El manifiesto declara Internet y estado de red; no se encontraron llamadas de red en la lógica revisada.
RNF-08	La interfaz debe usar un idioma uniforme.	Parcial	La interfaz está en español, pero los avisos de victoria y empate están en inglés.

2.4 Criterios de aceptación

1. Una partida nueva muestra un tablero vacío y empieza el turno humano.
2. Una casilla ocupada conserva su símbolo cuando se vuelve a pulsar.
3. Una jugada válida de `X` va seguida por una respuesta legal de `O`.
4. La IA bloquea una amenaza inmediata y, cuando existe una victoria forzada, puede completarla.
5. El sistema reconoce una partida empatada y bloquea el tablero tras terminar.
6. Nuevo limpia el estado y Salir cierra el proceso de la aplicación.
7. Las opciones de menú deben ejecutar sus acciones antes de considerar completo el requisito RF-08.

Capítulo 3

Diseño del sistema

3.1 Arquitectura

La aplicación utiliza un proyecto MAUI de estilo SDK con interfaz XAML y código C#. El punto de arranque se configura en `MauiProgram`; `App` crea la ventana, `AppShell` establece la página inicial y `MainPage` construye y coordina la interfaz del juego. `UnbeatableAIController` calcula la jugada sin manipular controles visuales.

Las cuatro figuras se generaron con Mermaid. Sus fuentes editables `01_arquitectura.mmd`, `02_casos_de_uso.mmd`, `03_secuencia_turno.mmd` y `04_diagrama_clases.mmd` se incluyen en el directorio `diagramas_mermaid`.

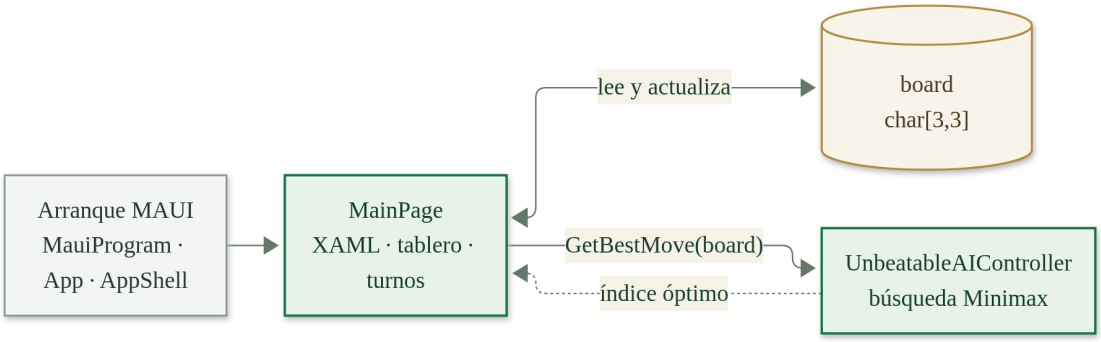


Figura 3.1 Arquitectura lógica de la aplicación MAUI.

3.2 Representación del tablero

La clase de página mantiene una matriz `char[3,3]` y un arreglo de nueve botones. Un índice lineal i se convierte a coordenadas usando fila $i/3$ y columna $i \bmod 3$. En XAML la columna se calcula con $i \bmod 3$ y la fila con $i/3$. La matriz y la interfaz se actualizan en el mismo movimiento.

Tabla 3.1 Relación entre índice lineal y coordenadas del tablero

Índices	Fila de matriz	Controles
0, 1, 2	0	1, 2, 3
3, 4, 5	1	4, 5, 6
6, 7, 8	2	7, 8, 9

3.3 Diagrama de casos de uso

El actor es una persona que inicia una partida, marca casillas, reinicia, abre los menús o sale. La IA es un servicio interno invocado desde el flujo de una jugada; no es un segundo usuario externo.

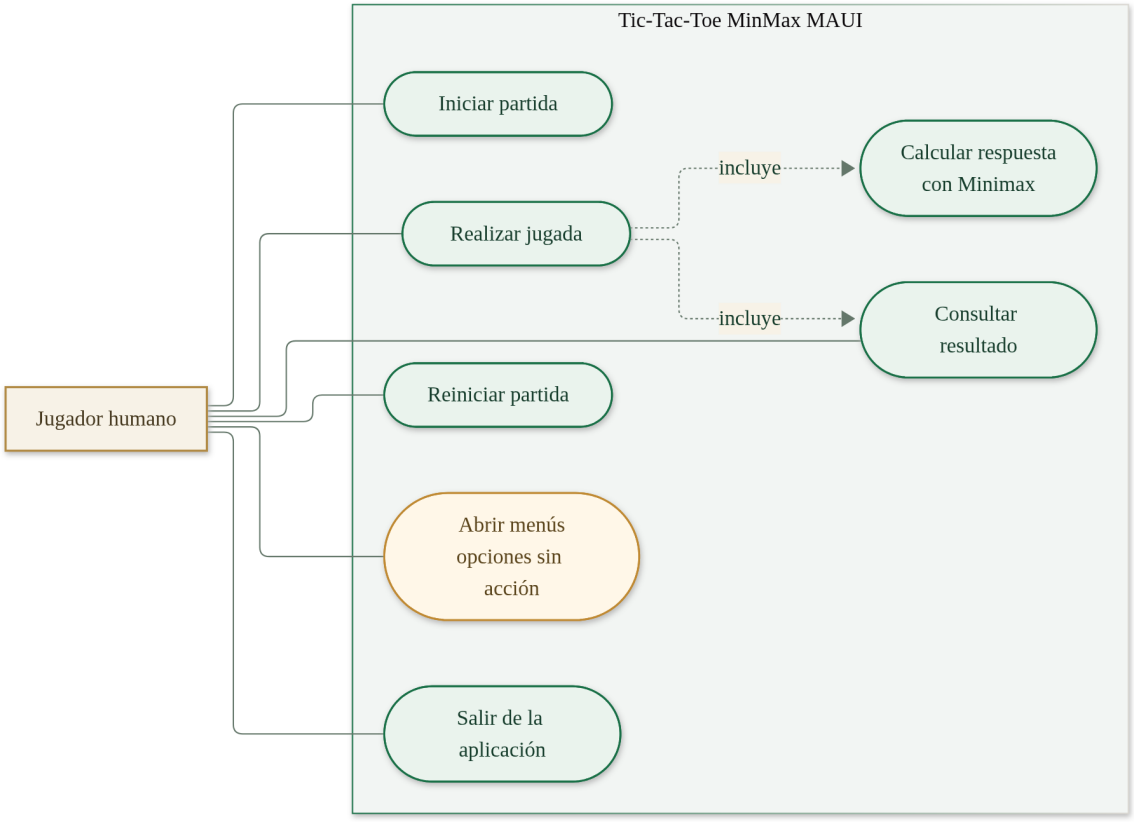


Figura 3.2 Casos de uso visibles. La selección de opciones internas de menú está pendiente.

3.3.1 Descripción resumida de casos de uso

Tabla 3.2 Casos de uso y condición observada

ID	Caso	Flujo principal
CU-01	Iniciar partida	La página inicializa el tablero vacío, habilita las casillas y entrega el primer turno a `X`.
CU-02	Realizar jugada	La persona toca una casilla disponible; el programa escribe `X`, revisa el fin de partida y solicita la jugada de `O`.
CU-03	Consultar resultado	Tras cada jugada se buscan líneas ganadoras y se comprueba si el tablero está lleno; si corresponde, se muestra un aviso.
CU-04	Reiniciar	El botón Nuevo borra la matriz y los textos de los botones, los habilita y devuelve el turno a la persona.
CU-05	Abrir menú	Archivo, Ver o Ayuda muestran una hoja de acciones. Elegir una opción no ejecuta una acción en el código actual.
CU-06	Salir	El botón Salir invoca el cierre de la aplicación.

3.4 Diagrama de secuencia de una jugada

3.5 Diagrama de clases

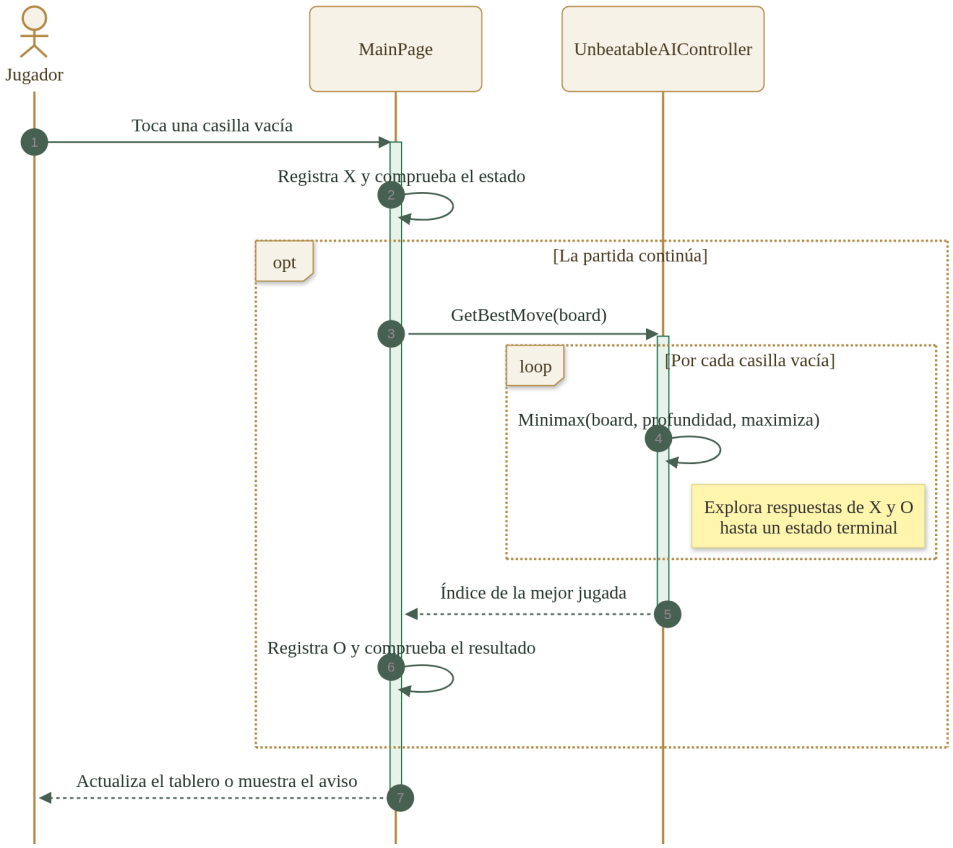


Figura 3.3 Secuencia de una jugada y búsqueda recursiva de la IA.

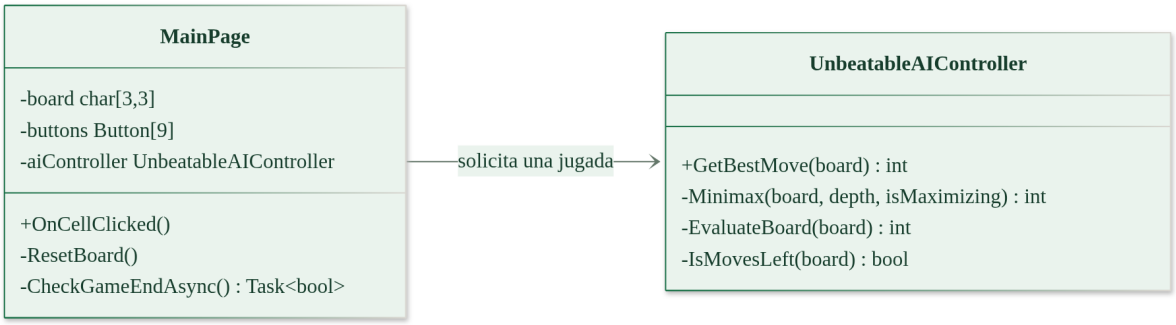


Figura 3.4 Clases centrales de la lógica del juego y dependencia de Minimax.

Capítulo 4

Algoritmo Minimax

4.1 Idea de decisión

Minimax es una búsqueda adversarial de suma cero: la IA elige el movimiento que maximiza su resultado y supone que la persona responderá con el movimiento que más lo reduce. El material didáctico de Stanford presenta el mismo principio de búsqueda recursiva sobre árboles de juego [7]. Para el tablero de tres en raya se exploran jugadas legales hasta alcanzar una victoria, una derrota o un empate.

La puntuación terminal del controlador es:

$$U(s,d) = \begin{cases} 10 - d, & \text{si gana la IA;} \\ -10 + d, & \text{si gana la persona;} \\ 0, & \text{si el tablero termina en empate.} \end{cases}$$

El término d favorece victorias rápidas y retrasa derrotas. En estados no terminales, la rutina no asigna una heurística intermedia: sigue expandiendo los movimientos hasta un resultado terminal.

4.2 Procedimiento implementado

Pseudocódigo de Minimax

```
GetBestMove(board):
    mejorPuntaje = -infinito
    mejorIndice = -1
    para cada indice i entre 0 y 8:
        si board[i] esta vacia:
            colocar O
            puntaje = Minimax(board, profundidad=0, maximiza=false)
            restaurar casilla vacia
            si puntaje > mejorPuntaje:
                guardar puntaje e indice
    devolver mejorIndice

Minimax(board, profundidad, maximiza):
    puntaje = evaluar ganador
    si gana O: devolver 10 - profundidad
    si gana X: devolver -10 + profundidad
    si no quedan movimientos: devolver 0
    si maximiza:
        probar cada casilla vacia como O
        devolver el maximo de las llamadas recursivas
    si no:
        probar cada casilla vacia como X
        devolver el minimo de las llamadas recursivas
```

Fragmento C# del controlador Minimax

```
private int Minimax(char[,] board, int depth, bool isMaximizing)
{
    int score = EvaluateBoard(board);
    if (score == 10) return score - depth;
    if (score == -10) return score + depth;
    if (!IsMovesLeft(board)) return 0;
    // Las ramas restantes prueban cada casilla vacia con MAX o MIN.
}
```

En `GetBestMove`, cada posible movimiento de `O` se simula temporalmente en la misma matriz recibida. Después de cada llamada recursiva se restaura el espacio vacío, para que las siguientes alternativas empiecen desde el mismo estado. Como la primera llamada recursiva usa `isMaximizing=false`, la IA supone que después de su movimiento la persona elegirá la respuesta más perjudicial para ella.

4.3 Condiciones terminales y desempate

`EvaluateBoard` examina las tres filas, tres columnas y dos diagonales. Devuelve 10 cuando encuentra una línea de `O`, -10 para una línea de `X` y 0 en caso contrario. `Minimax` distingue entre ese cero no terminal

y un empate real mediante `IsMovesLeft`: solo cuando no quedan casillas devuelve el valor terminal cero.

Si dos movimientos reciben la misma puntuación, el controlador conserva el primero que encuentra porque actualiza la mejor jugada solo cuando `score > bestScore`. El orden es el índice lineal de 0 a 8. Esto produce desempates deterministas, aunque no prioriza explícitamente centro o esquinas.

4.4 Costo de búsqueda

Sea b el máximo número de movimientos disponibles en un estado y d la profundidad restante. La búsqueda completa tiene costo temporal del orden de $O(b^d)$; aquí $b \leq 9$ y $d \leq 9$. La recursión es de profundidad acotada y modifica una única matriz de 3×3, por lo que el espacio auxiliar crece con la profundidad, $O(d)$, además del tablero de tamaño fijo. El código no aplica poda alfa-beta ni memorización. Para tres en raya el espacio de búsqueda es pequeño; esta elección mantiene simple la explicación del algoritmo.

La propiedad de no perder se deriva del cálculo óptimo en estados legales y de que ambos resultados terminales se evalúan correctamente. Las capturas existentes demuestran escenarios seleccionados —por ejemplo, bloquear una columna y completar una diagonal—; no sustituyen una enumeración exhaustiva de todos los estados posibles.

Capítulo 5

Desarrollo con SRUM+XP

5.1 Adaptación de la metodología

La guía del laboratorio solicita SRUM+XP y menciona diseño, desarrollo y pruebas [1]. El repositorio contiene el código y las evidencias de ejecución, pero no contiene una bitácora de sprints, un tablero histórico, acuerdos de programación en pareja ni registros de integración continua. Por esta razón, la tabla siguiente organiza el trabajo por incrementos comprobables y no atribuye fechas, roles o ceremonias que no están documentados.

Tabla 5.1 Incrementos reconstruidos a partir del proyecto

Fase	Objetivo	Resultado en el código	Artefacto
Diseño	Separar interfaz, estado y decisión de IA.	Página XAML para presentación; controlador separado para búsqueda.	Diagramas de este informe.
Tablero	Construir tablero y turnos.	Matriz 3×3, nueve botones, validación de casilla y reinicio.	`MainPage.xaml.cs`.
Minimax	Integrar decisión adversarial.	Simulación de jugadas y búsqueda Minimax recursiva.	`UnbeatableAIController.cs`.
Pruebas	Revisar escenarios en Android.	Automatización por ADB/UIAutomator, capturas PNG y registro JSON.	14 imágenes, 25 registros.
Informe	Entregar requisitos, análisis y resultados.	Especificación, diagramas, configuración y métricas.	Este informe.

5.2 Prácticas XP observables

Tabla 5.2 Relación entre prácticas ágiles y evidencia encontrada

Práctica	Observación	Estado
Diseño simple	La búsqueda está aislada en una clase y usa una matriz pequeña, sin dependencias de interfaz.	Evidenciado en el código.
Pruebas automatizadas	El script existente consulta la jerarquía UI, toca controles, comprueba el tablero y guarda hashes de imágenes.	Evidencia presente; no se ejecutó durante la elaboración del informe.
Retroalimentación frecuente	Los resultados incluyen capturas de cada escenario y un JSON legible por máquina.	Evidencia presente.
Programación en pareja	No se encontró registro de sesiones o roles de pareja.	No verificable.
Integración continua	No se encontró pipeline ni configuración de compilación automatizada.	No verificado.
Refactorización continua	El historial de cambios no está disponible en el directorio de trabajo.	No verificable.

5.3 Backlog de mejoras priorizado

1. **Prioridad alta:** conectar el resultado de cada menú con su acción. Nuevo y Salir pueden reutilizar los manejadores existentes; Puntajes requiere definir y almacenar un modelo de puntuación; Ayuda requiere contenido.
2. **Prioridad alta:** añadir pruebas unitarias del controlador Minimax para victoria inmediata, bloqueo obligatorio, empate, tablero lleno y ausencia de movimiento.
3. **Prioridad media:** unificar el idioma de los avisos y los textos; agregar una presentación accesible del turno y del estado terminal.
4. **Prioridad media:** medir el tiempo de decisión en un dispositivo y conservar la medición como evidencia de rendimiento.
5. **Prioridad baja:** evaluar poda alfa-beta o memorización si el tablero se generaliza a juegos con mayor espacio de búsqueda.

Capítulo 6

Implementación y configuración en Visual Studio

6.1 Estructura del proyecto

Tabla 6.1 Archivos de entrada más relevantes

Archivo	Responsabilidad observada
`TicTacToeMinMax.csproj`	Destino Android, identidad de aplicación, recursos e inclusiones de paquetes.
`MauiProgram.cs`	Construye la aplicación MAUI y agrega registro de depuración en configuración DEBUG.
`App.xaml.cs`	Crea la ventana principal con `AppShell`.
`AppShell.xaml`	Declara `MainPage` como contenido inicial.
`MainPage.xaml`	Define textos, tablero contenedor, botones, menús y estilos.
`MainPage.xaml.cs`	Coordina turnos, estado, eventos, reinicio, salida y resultados.
`UnbeatableAIController.cs`	Evalúa estados y busca el movimiento de `O`.
`AndroidManifest.xml`	Manifiesto y permisos Android declarados.
`launchSettings.json`	Incluye un perfil llamado *Windows Machine*; no identifica el emulador Android.

6.2 Configuración comprobada

La inspección del equipo y del proyecto encontró Visual Studio Community 2026 versión 18.11.0, SDK de .NET 10.0.402 y las cargas de trabajo de MAUI/Android 10.0.110. El proyecto usa MAUI con destino Android y mínimo de plataforma API 21. En la barra de Visual Studio se ve la configuración *Debug*, *Any CPU* y el destino seleccionado *Pixel 7 – API 36.0 (Android 16.0 – API 36)*. Los requisitos de plataforma de MAUI para Android 5.0/API 21 o posterior coinciden con el valor mínimo del proyecto [2].

Tabla 6.2 Propiedades del proyecto y valores encontrados

Propiedad	Valor observado
Target framework	`net10.0-android`
Tipo de salida	`Exe`
MAUI y proyecto único	`UseMaui=true`; `SingleProject=true`
Nullable reference types	Habilitado
Versión mínima Android	`SupportedOSPlatformVersion=21.0`
Título de aplicación	`Tic-Tac-Toe`
Application ID	`com.companyname.tictactoeminmax`
Versión de aplicación	`1.0`; versión numérica `1`
Icono e imagen de inicio	`Resources/AppIcon/appicon.png`; pantalla inicial blanca, base 128×128
Imagen incluida	`Resources/Images/pic_character.png`
Paquetes	`Microsoft.Maui.Controls` con `$(MauiVersion)` y `Microsoft.Extensions.Logging.Debug` 10.0.0

El proyecto incluye una sola plataforma en `TargetFramework`, aunque MAUI permita compartir código con otros destinos. El emulador seleccionado en Visual Studio es evidencia de configuración de depuración; no se interpreta como una ejecución nueva de la aplicación. El README de las capturas identifica el dispositivo usado para la evidencia anterior como `emulator-5554`; no se atribuye un número de serie a la selección mostrada por Visual Studio.

6.3 Capturas de configuración en Visual Studio

Las capturas se generaron desde la ventana de Visual Studio con el proyecto MAUI cargado. Se muestran el destino Android seleccionado y la configuración del archivo de proyecto; no se modificó el código para producirlas.

6.4 Pasos para abrir y ejecutar desde Visual Studio

1. Iniciar Visual Studio y escoger **Abrir un proyecto o una solución**.
2. Seleccionar `TicTacToeMinMax.csproj` dentro de `Tic-Tac-Toe/MinMax/C#_MAUI_Mobile`. El directorio no contiene un archivo de solución para este proyecto MAUI; se abre directamente el proyecto.
3. Esperar la restauración de paquetes. Para desarrollar MAUI en Windows deben estar instalados el SDK de .NET MAUI y los componentes Android; Microsoft documenta su instalación desde Visual Studio Installer y la verificación mediante las cargas de trabajo instaladas [4].

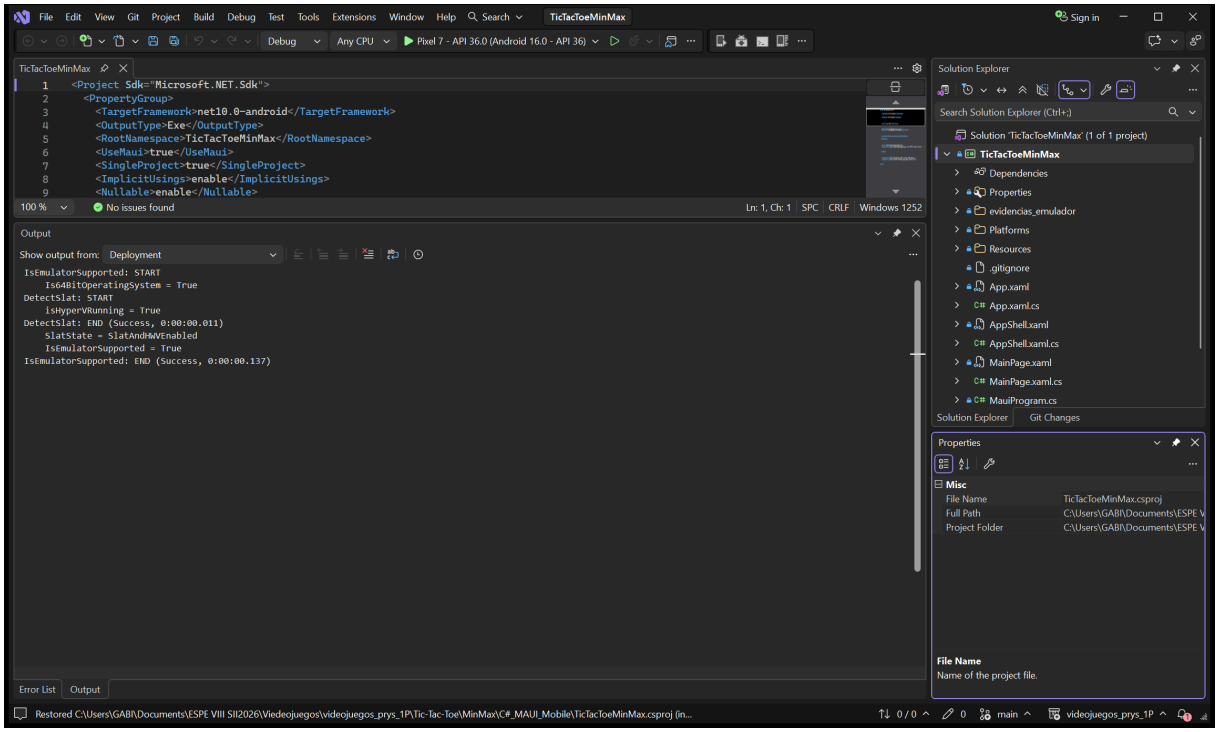


Figura 6.1 Visual Studio Community 2026 con el proyecto cargado. La barra muestra Debug, Any CPU y el emulador Android seleccionado; el editor muestra el destino `net10.0-android`.

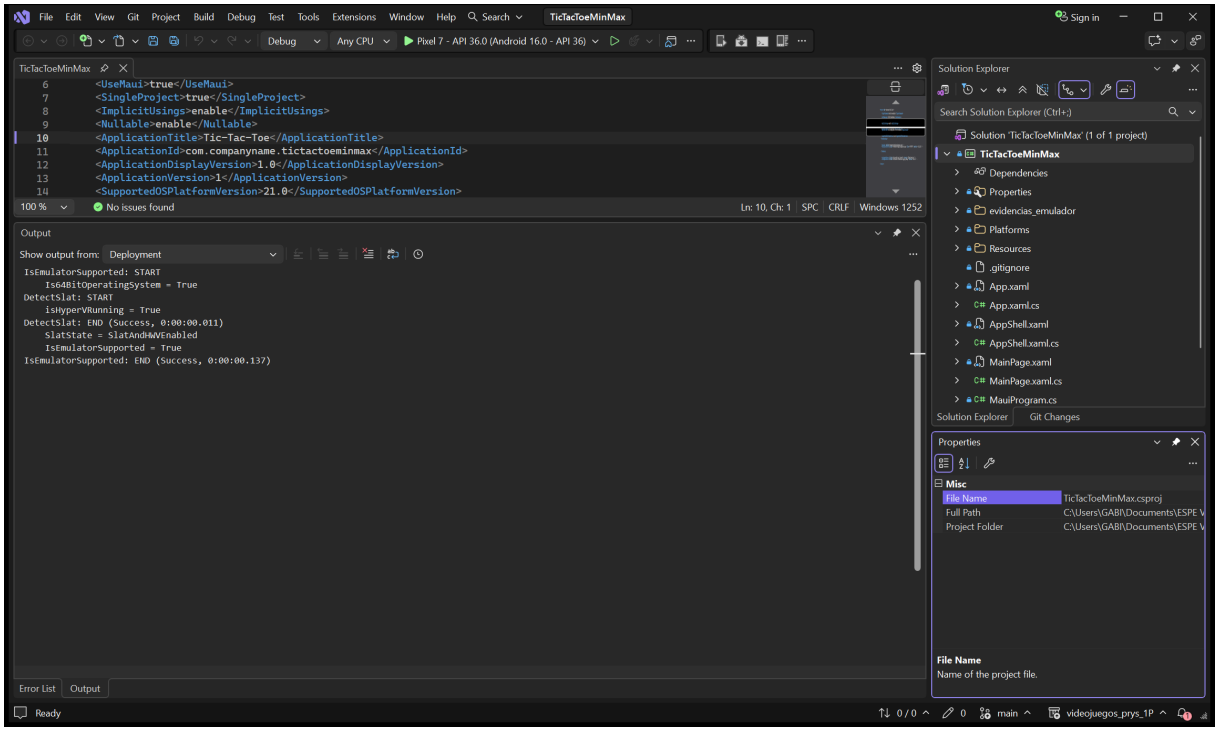


Figura 6.2 Configuración de aplicación y plataforma en el `.csproj`: título, Application ID, versiones y mínimo Android API 21.

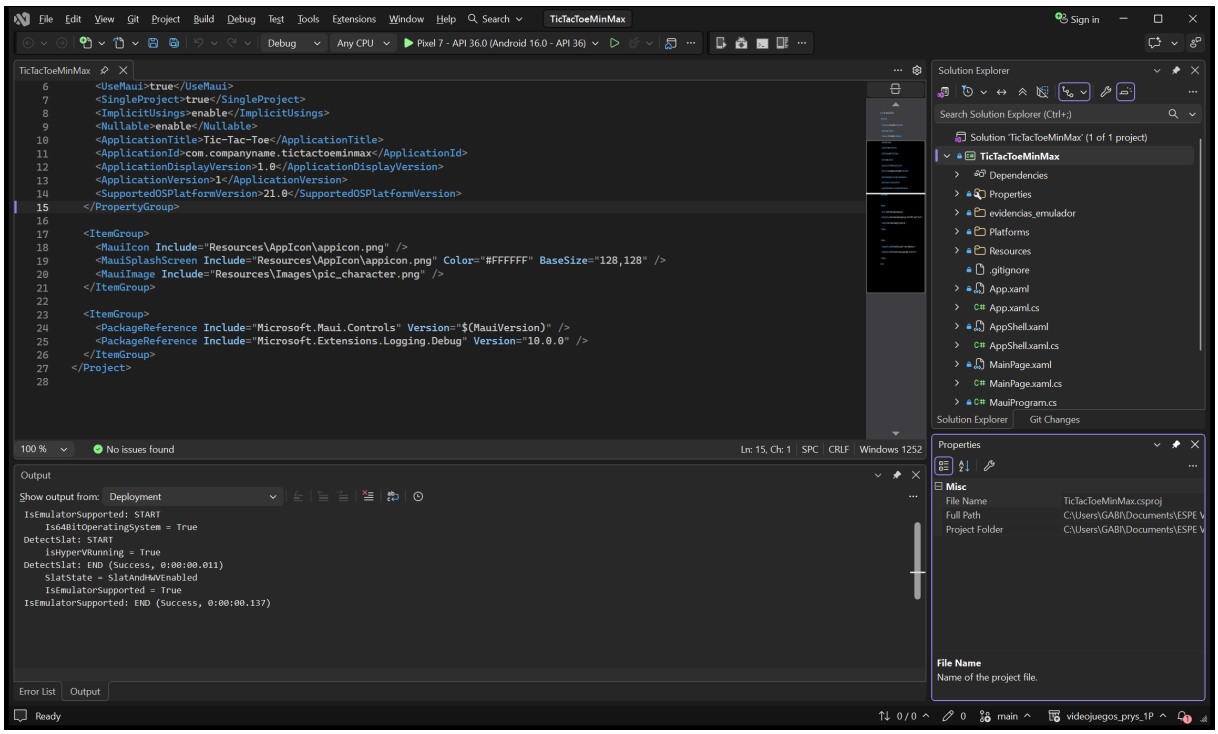


Figura 6.3 Recursos MAUI y referencias de paquetes declarados en el proyecto.

4. En la barra de depuración, seleccionar **Debug**, **Any CPU** y un emulador Android. La captura de este equipo muestra **Pixel 7 – API 36.0 (Android 16.0 – API 36)**.

5. Usar **Iniciar** o presionar **F5**. Si Visual Studio solicita un dispositivo, elegir un emulador Android disponible y esperar el despliegue.

6. En la aplicación, tocar una casilla vacía. Comprobar que la persona juega con `X` y que la respuesta de Minimax aparece con `O`.

7. Usar Nuevo para limpiar el tablero. La opción Salir cierra la aplicación; las opciones de los menús Archivo, Ver y Ayuda son solo presentacionales en la versión revisada.

6.5 Configuración Android que conviene conservar

Las cargas de trabajo y el destino Android son coherentes con el archivo del proyecto. El perfil de `launchSettings.json` llamado *Windows Machine* no describe el dispositivo Android; la selección del emulador se realiza desde la barra de depuración de Visual Studio. El manifiesto declara `INTERNET` y `ACCESS_NETWORK_STATE`; al no encontrarse uso de red en la lógica del juego, conviene justificar o retirar esos permisos en una revisión de configuración.

Capítulo 7

Validación y métricas

7.1 Evidencia de ejecución existente

La carpeta `evidencias_emulador/capturas_20261104_175826` conserva 14 capturas PNG, un README, un archivo `resultados.json` con hashes SHA-256 y un script de automatización basado en ADB y UIAutomator. El README registra 25 verificaciones con estado PASS. La revisión del JSON y del README muestra que ese estado debe leerse junto con el detalle de cada caso: en las opciones internas de menú, PASS significa que la selección no alteró la partida, porque el menú carece de manejadores, no que la opción anunciada funcione.

La evidencia incluye el inicio, turno humano e IA, selección de casilla ocupada, menús, bloqueo de una amenaza, victoria de la IA, bloqueo tras finalizar, reinicio, empate, salida y reapertura. El script evita crear una victoria humana artificial: la propia IA bloquea las jugadas ganadoras de `X`. Por eso el paquete de evidencia no contiene una captura de victoria humana. Este comportamiento es coherente con la finalidad de Minimax, pero no reemplaza una prueba unitaria de todos los estados.

El registro identifica `02_turno_humano_ia.png` y `03_casilla_ocupada.png` con el mismo hash. La segunda captura documenta el paso de prueba pero su imagen es idéntica a la anterior; debe contarse como una limitación de la evidencia visual. Este informe conserva ambas con sus nombres originales para mantener correspondencia con el JSON.

Tabla 7.1 Resumen de escenarios de aceptación documentados

Escenario	Resultado registrado	Lectura
Inicio, jugada de `X` y respuesta de `O`	PASS	Evidencia visible de la respuesta Minimax.
Casilla ocupada	PASS	El tablero no cambia; la imagen asociada duplica el paso previo.
Bloqueo de amenaza inmediata	PASS	`O` completa el bloqueo de la columna amenazada.
Victoria de la IA y fin de partida	PASS	Se presenta el aviso y no se admite una nueva jugada.
Empate	PASS	Se completa el tablero sin línea ganadora y aparece el aviso.
Nuevo, Salir y reapertura	PASS	Los botones reinician y cierran; la reapertura empieza vacía.
Opciones internas de los menús	PASS en el registro	La selección se conserva sin ejecutar una acción; RF-08 sigue pendiente.

7.2 Métricas estáticas

El conteo se realizó sobre los archivos fuente actuales, sin incluir `bin`, `obj` ni las evidencias. "Líneas físicas" incluye líneas vacías y comentarios; se informa así para que la métrica no parezca una medida de complejidad o productividad.

Tabla 7.2 Métricas observables del proyecto

Métrica	Valor	Interpretación
Archivos C# de la aplicación	7	Incluye página, arranque y archivos Android.
Líneas físicas C#	359	Recuento de líneas, incluidas vacías y comentarios.
Líneas físicas en `MainPage.xaml.cs`	182	Coordinación de turnos e interfaz.
Líneas físicas en `UnbeatableAIController.cs`	121	Búsqueda Minimax y evaluación terminal.
Líneas físicas en `MainPage.xaml`	64	Estructura visual de la página.
Destino de compilación declarado	1	`net10.0-android`.
Capturas de ejecución existentes	14	PNG conservados en el paquete de evidencias.
Verificaciones listadas en el JSON	25	Todas aparecen como PASS; seis selecciones de menú confirman que no hay acción asociada.
Líneas de prueba automatizada	No sumadas	El script existente se considera evidencia, no código de producción.

7.3 Observaciones sobre la calidad de la evidencia

- La automatización existente consulta nodos de UI por texto e identificador, lo cual permite comprobar casillas y acciones visibles sin inspeccionar solo píxeles.
- El script guarda hashes para detectar cambios en las capturas. El hash idéntico de las capturas 02 y 03 muestra que ese control también ayuda a encontrar evidencia repetida.
- El script contiene una ruta absoluta de ADB bajo el perfil `C:\Users\usuario01`. Esa ruta no coincide con el perfil de usuario del entorno actual y reduce la portabilidad; conviene leer el SDK desde una variable de entorno o una configuración local.
- La evidencia existente proviene de `emulator-5554`. La captura actual de Visual Studio muestra un emulador Pixel 7 con API 36 seleccionado, pero no prueba que se haya vuelto a ejecutar la matriz de escenarios bajo esa selección.
- En esta elaboración no se volvió a ejecutar el script, no se realizó una compilación de la aplicación y no se midió el tiempo de respuesta. La sección reporta los artefactos que ya existían y la inspección estática del código.

Capítulo 8

Análisis, limitaciones y conclusiones

8.1 Análisis

La separación entre página y controlador hace que la decisión de la IA no dependa de botones ni de XAML. La página controla las reglas de interacción y la clase del algoritmo recibe solo la matriz del tablero. Para el dominio pequeño de tres en raya, una búsqueda completa sin poda es suficientemente directa y fácil de inspeccionar.

La evidencia seleccionada cubre varios resultados relevantes, pero no certifica la corrección de todos los estados posibles. La verificación del comportamiento óptimo se fortalece con pruebas unitarias del controlador que recorran estados construidos y validen la jugada esperada. También conviene comparar el resultado de Minimax con un enumerador independiente de todos los tableros legales.

8.2 Limitaciones detectadas

1. Las opciones de los menús muestran nombres pero no conectan la selección con acciones; puntajes y ayuda no existen como funciones terminadas.
2. Los avisos de victoria y empate usan inglés mientras el resto de la pantalla está en español.
3. No se guarda historial de partidas ni puntuación.
4. El proyecto está configurado únicamente para Android en su `TargetFramework`; no debe describirse como una entrega ya ejecutable en todas las plataformas MAUI.
5. Los permisos de red del manifiesto no se justifican en el flujo de juego revisado.
6. El script de evidencias usa una ruta local fija de ADB, y al menos dos imágenes son idénticas según sus hashes.
7. No se encontraron registros que permitan afirmar qué estudiante implementó cada parte, cuántas iteraciones hubo o que se practicó programación en pareja.

8.3 Conclusiones

1. El proyecto implementa una versión funcional de Tic-Tac-Toe para Android con tablero de tres por tres, turnos de `X` y `O`, validación de casillas, detección de victoria y empate, reinicio y salida.
2. El controlador aplica Minimax de forma recursiva: maximiza las puntuaciones de `O`, minimiza las respuestas de `X`, valora la rapidez de victoria y la demora de derrota, y devuelve un movimiento determinista ante empates de puntuación.

3. La configuración observada en Visual Studio es consistente con el destino del proyecto: .NET 10, MAUI para Android, API mínima 21 y un emulador Pixel 7 API 36 seleccionado para depuración.

4. Las 14 capturas y 25 registros existentes apoyan la validación de escenarios concretos. Los detalles del registro revelan que las opciones internas de los menús no ejecutan acciones; ese requisito debe permanecer marcado como pendiente.

5. El siguiente incremento recomendado es completar las acciones de menú y añadir pruebas unitarias del algoritmo. Luego conviene revisar permisos, idioma, portabilidad del script y mediciones de rendimiento.

8.4 Recomendación de entrega

Antes de entregar el informe, completar en la carátula los nombres de estudiantes y el número de grupo de trabajo. La guía deja esos campos como datos del grupo y el directorio no contiene nombres que permitan completarlos con certeza.

Referencias

[1] Universidad de las Fuerzas Armadas – ESPE, Departamento de Ciencias de la Computación. *Informe-Lab-Desarrollo-Videojuegos*. Guía de laboratorio para Desarrollo de Videojuegos, Unidad 1, documento DOCX proporcionado en el directorio de trabajo, 2026.

[2] Microsoft Learn. *Supported platforms for .NET MAUI apps*. `https://learn.microsoft.com/en-us/dotnet/maui/supported-platforms?view=net-maui-10.0` (consulta: 4 de noviembre de 2026).

[3] Microsoft Learn. *Target multiple platforms from .NET MAUI single project*. `https://learn.microsoft.com/en-us/dotnet/maui/fundamentals/app-startup?view=net-maui-10.0` (consulta: 4 de noviembre de 2026).

[4] Microsoft Learn. *Install .NET MAUI development tools on Windows*. `https://learn.microsoft.com/en-us/dotnet/maui/get-started/installation?view=net-maui-10.0` (consulta: 4 de noviembre de 2026).

[5] IEEE Standards Association. *IEEE 830-1998 – IEEE Recommended Practice for Software Requirements Specifications*. Estado del estándar y referencia a su sustitución. `https://standards.ieee.org/ieee/830/1222/` (consulta: 4 de noviembre de 2026).

[6] IEEE Standards Association. *IEEE/ISO/IEC 29148-2018 – Systems and software engineering – Life cycle processes – Requirements engineering*. `https://standards.ieee.org/ieee/802.1Q/6937/` (consulta: 4 de noviembre de 2026).

[7] Stanford University, CS106B. *Backtracking and Games: The Minimax Algorithm*. Material docente sobre búsqueda en árboles de juego. `https://cs.stanford.edu/people/eroberts/courses/cs106b/handouts/17-BacktrackingAndGames.pdf` (consulta: 4 de noviembre de 2026).

[8] Proyecto local *Tic-Tac-Toe MinMax MAUI*: archivos `TicTacToeMinMax.csproj`, `MainPage.xaml`, `MainPage.xaml.cs`, `MauiProgram.cs`, `UnbeatableAIController.cs` y recursos de Android. Código revisado en el directorio de trabajo, 2026.

[9] Proyecto local *Tic-Tac-Toe MinMax MAUI*: `evidencias_emulador/capturas_20261104_175826/README.md`, `resultados.json` y 14 capturas PNG. Evidencia previa incluida en el directorio de trabajo, 2026.

Anexo A

Evidencias de la aplicación en el emulador

Las figuras de este apéndice son las capturas originales del directorio de evidencias. Se conservaron sus nombres para que coincidan con `resultados.json`. El archivo 03 tiene el mismo hash que el 02 en el registro existente.

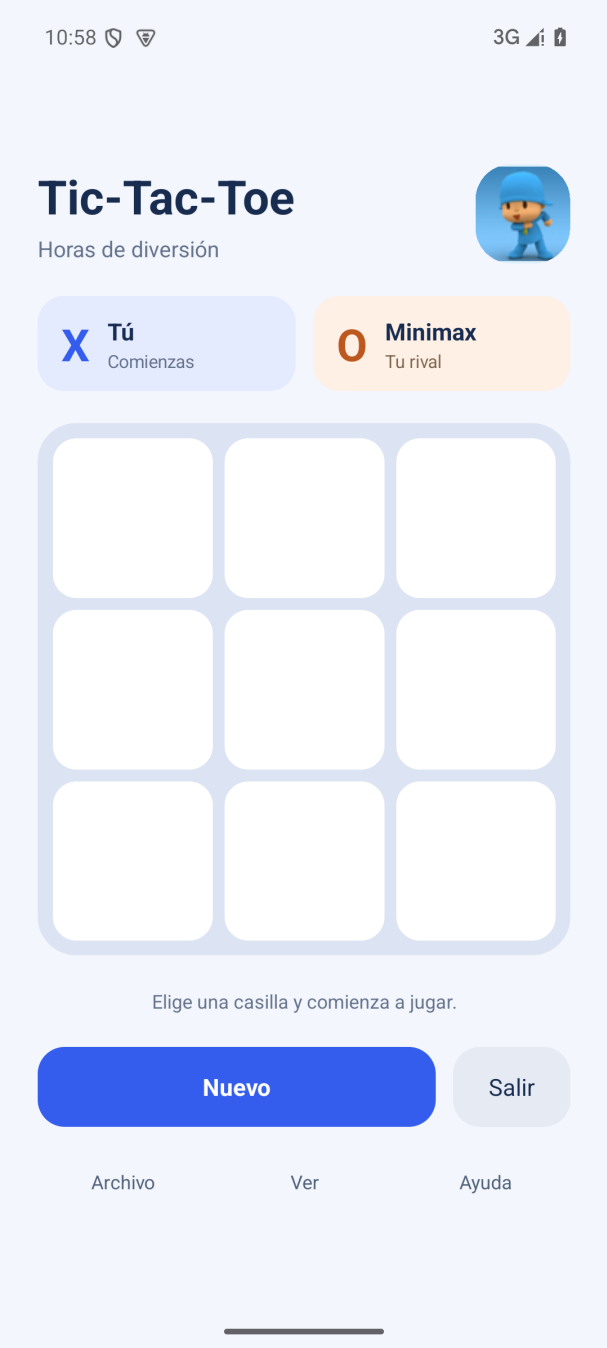


Figura A.1 Pantalla inicial con el tablero vacío, los símbolos de cada participante y los controles visibles.

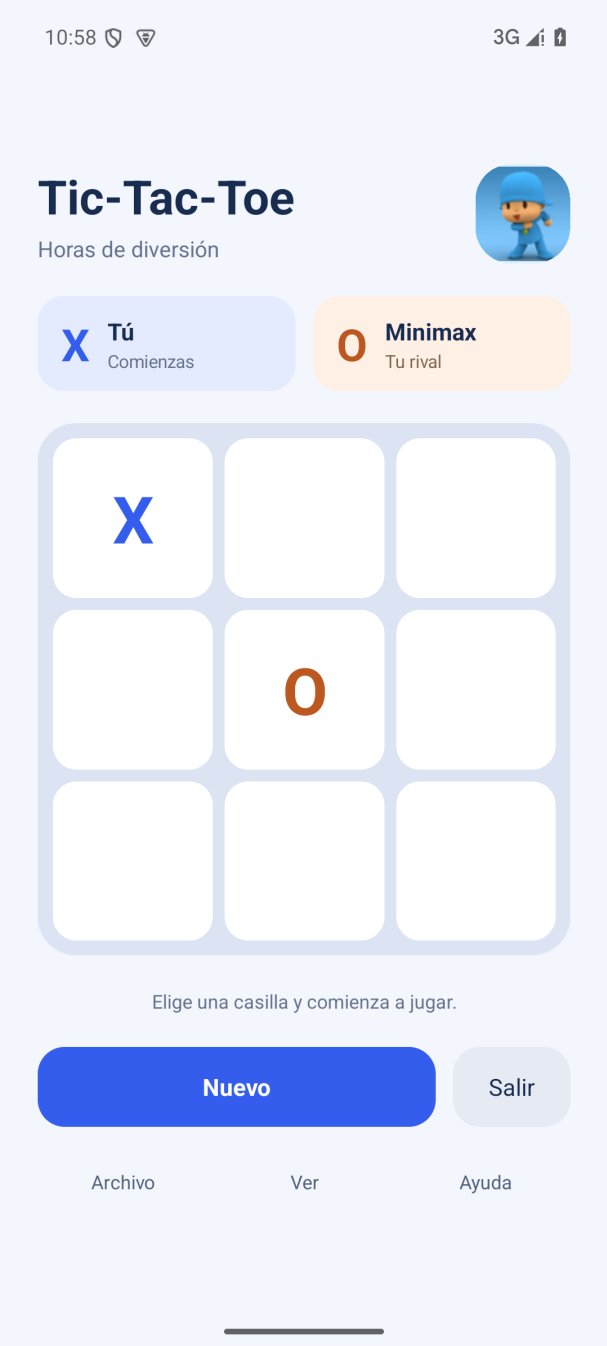


Figura A.2 La persona coloca X y Minimax responde con O.

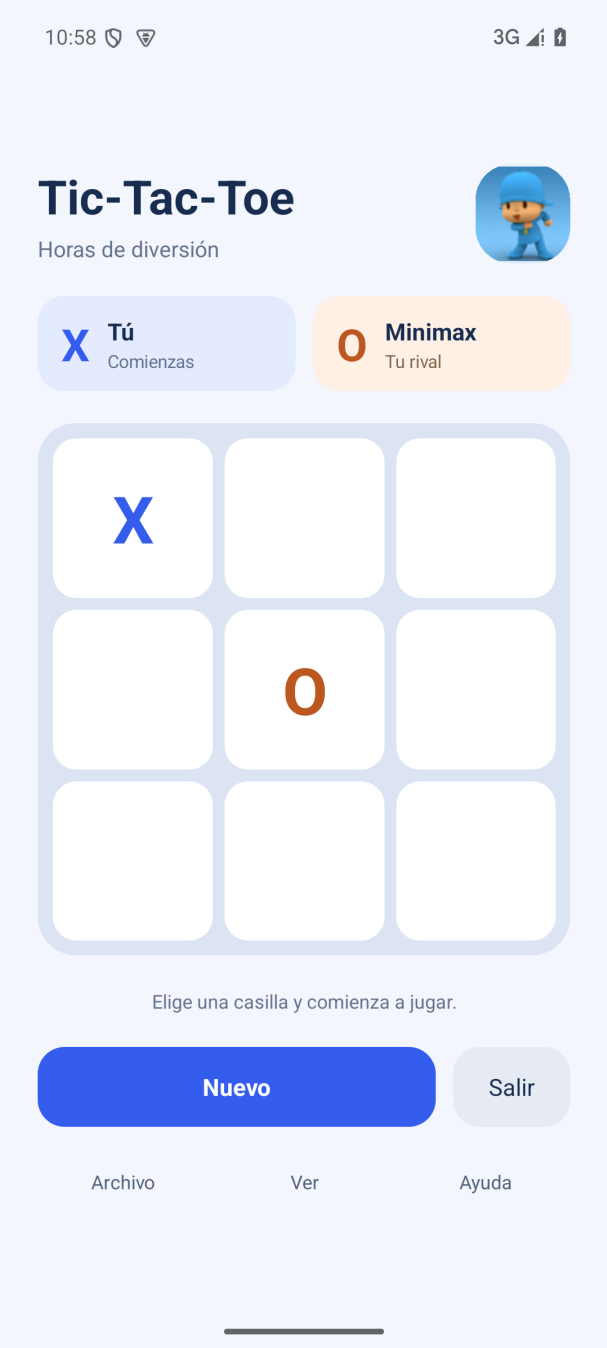


Figura A.3 Verificación registrada para la casilla ocupada; la imagen conserva el mismo hash que la captura 02.



Figura A.4 Hoja de acciones del menú Archivo. Las opciones no están conectadas a manejadores de acción.



Figura A.5 Hoja de acciones del menú Ver. Puntajes y Resetear puntajes son etiquetas visibles, sin función implementada.



Figura A.6 Hoja de acciones del menú Ayuda; las selecciones no abren contenido.

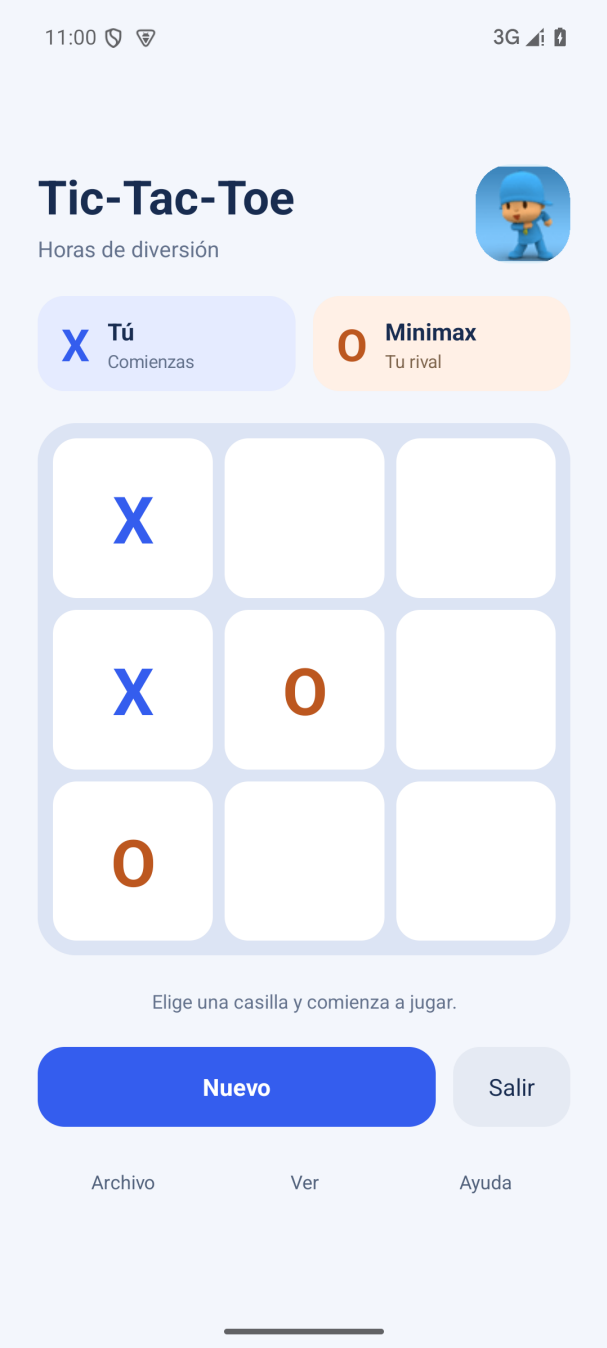


Figura A.7 Minimax bloquea la amenaza de victoria de X en la primera columna.

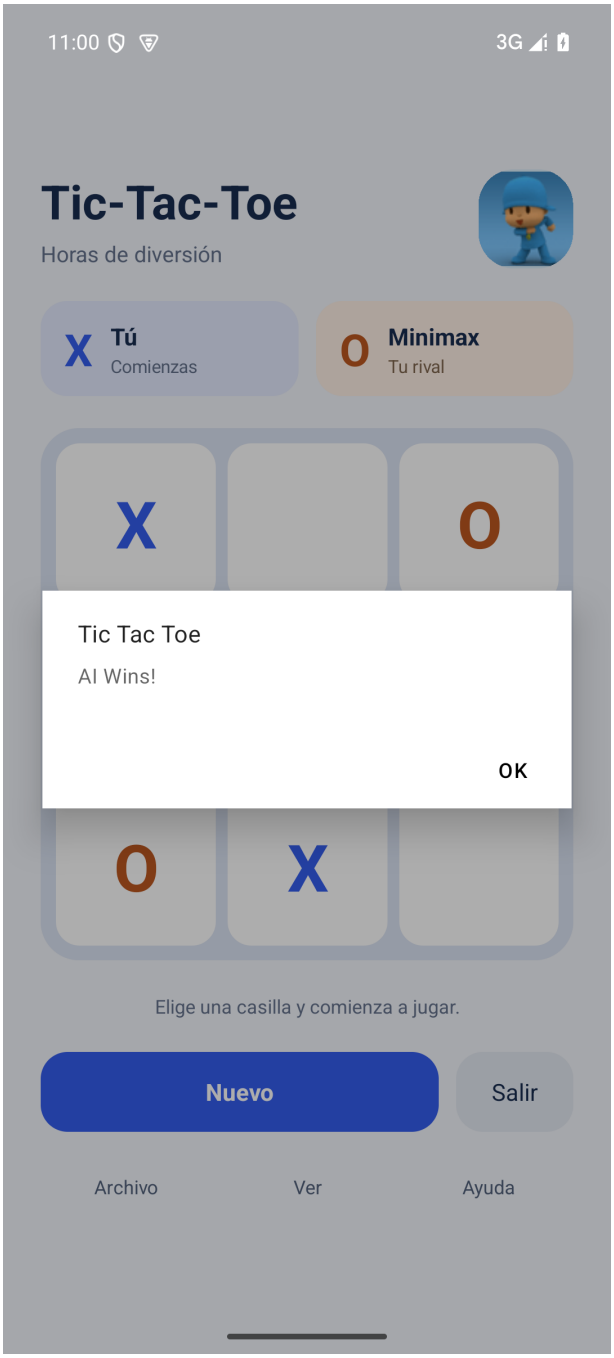


Figura A.8 Aviso de victoria de la IA registrado en el escenario de prueba.

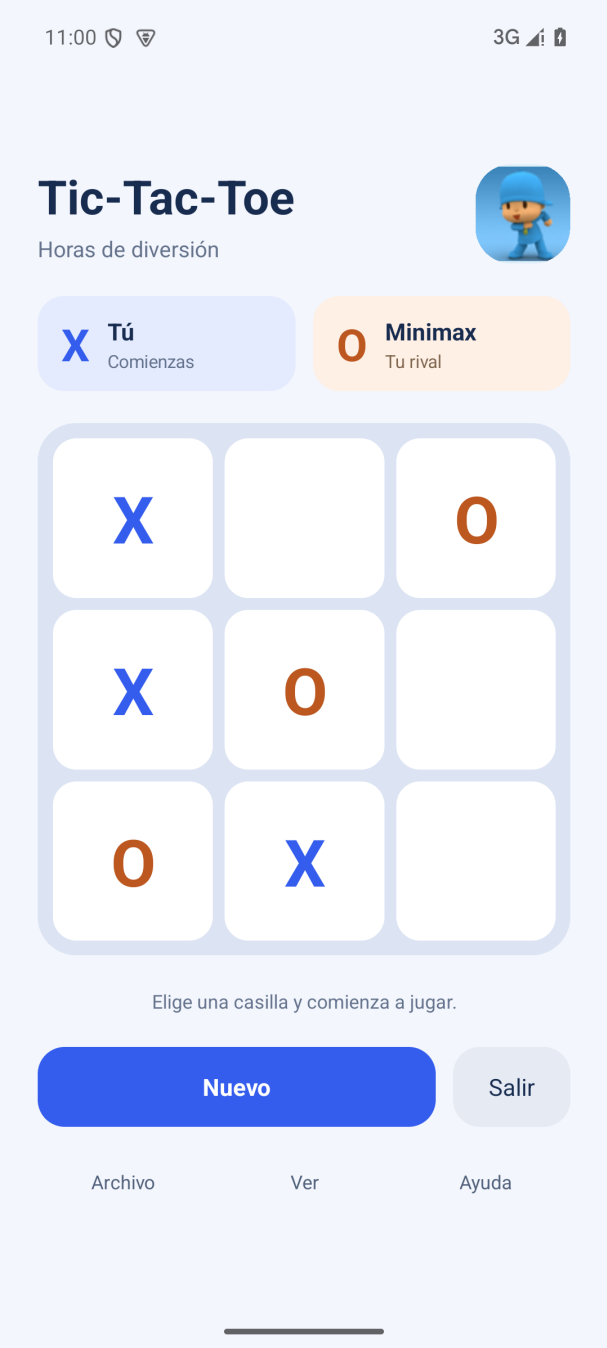


Figura A.9 Tablero con victoria diagonal de O; la entrada posterior queda bloqueada.

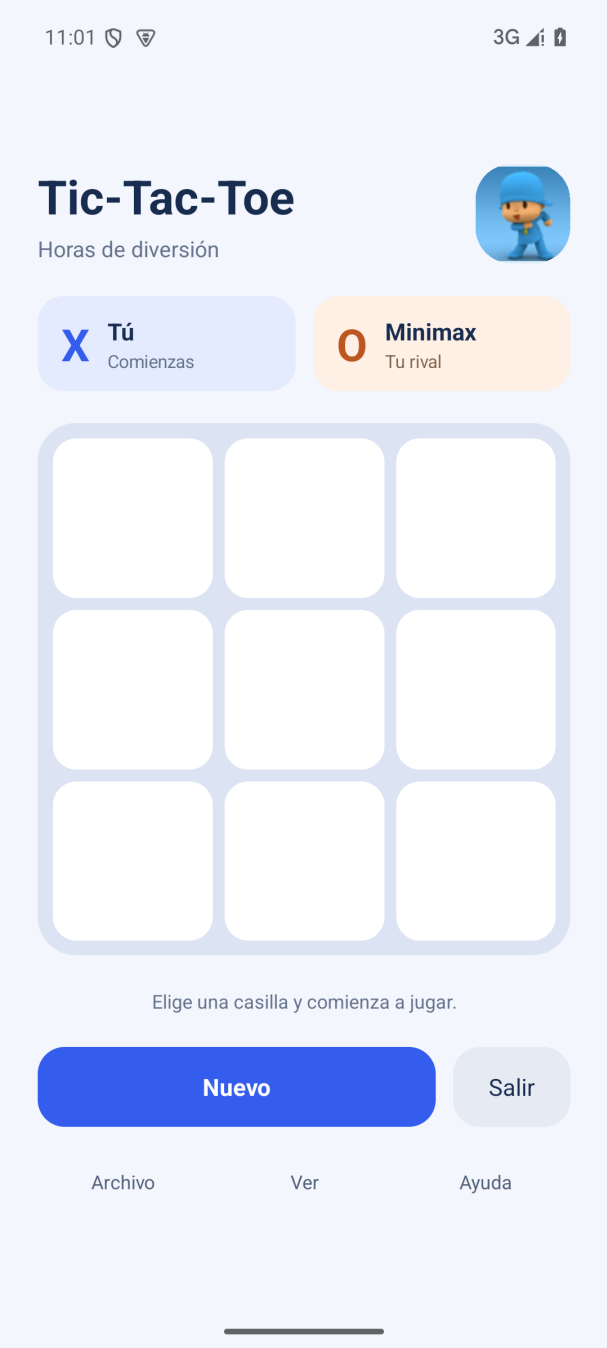


Figura A.10 El botón Nuevo limpia la partida y habilita nuevamente las casillas.

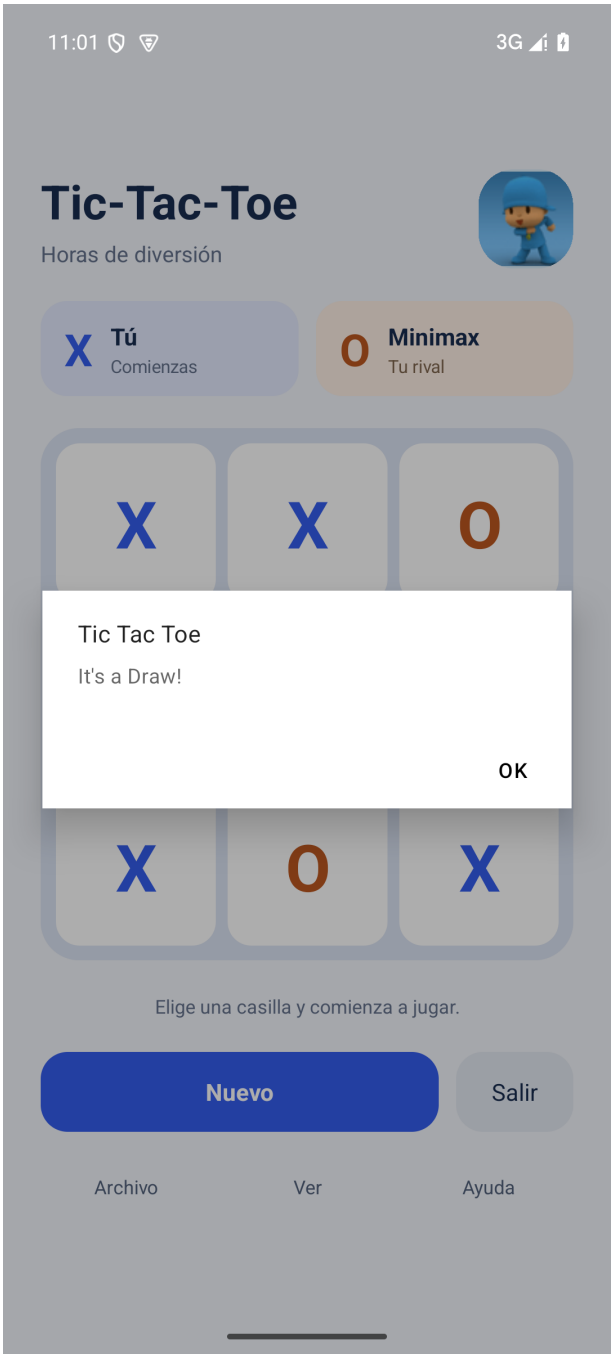


Figura A.11 Aviso de empate al completar el tablero sin una línea ganadora.

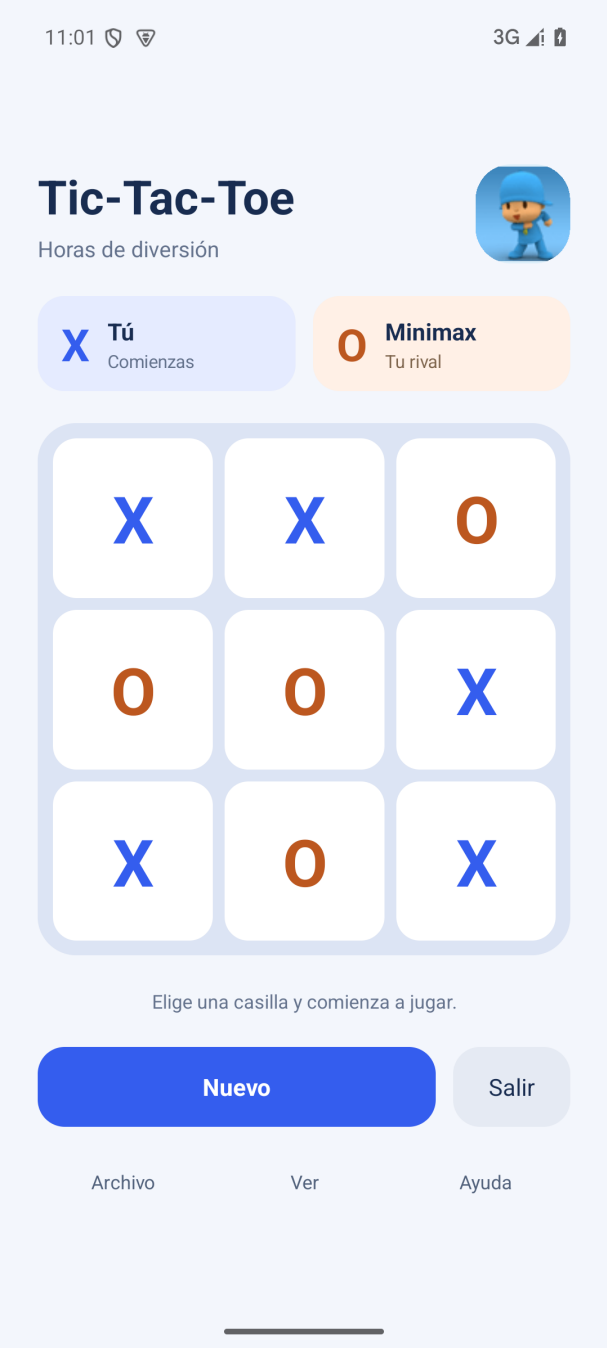


Figura A.12 Estado final del tablero empatado.

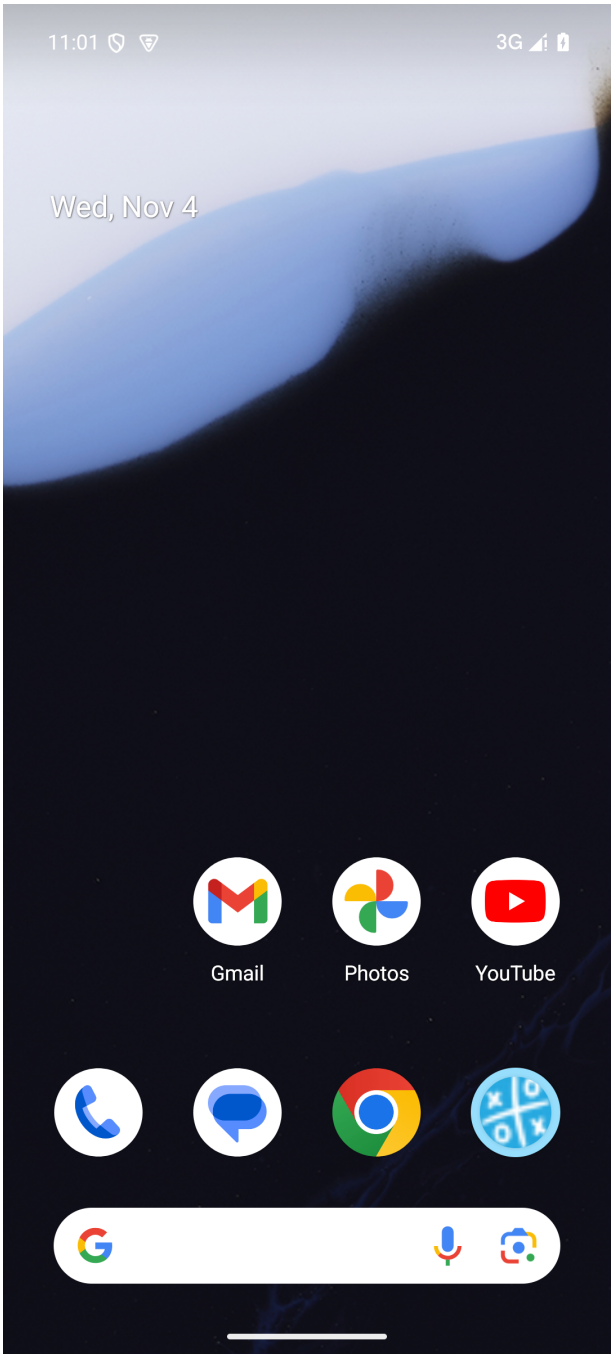


Figura A.13 Pantalla de inicio de Android tras usar el botón Salir.

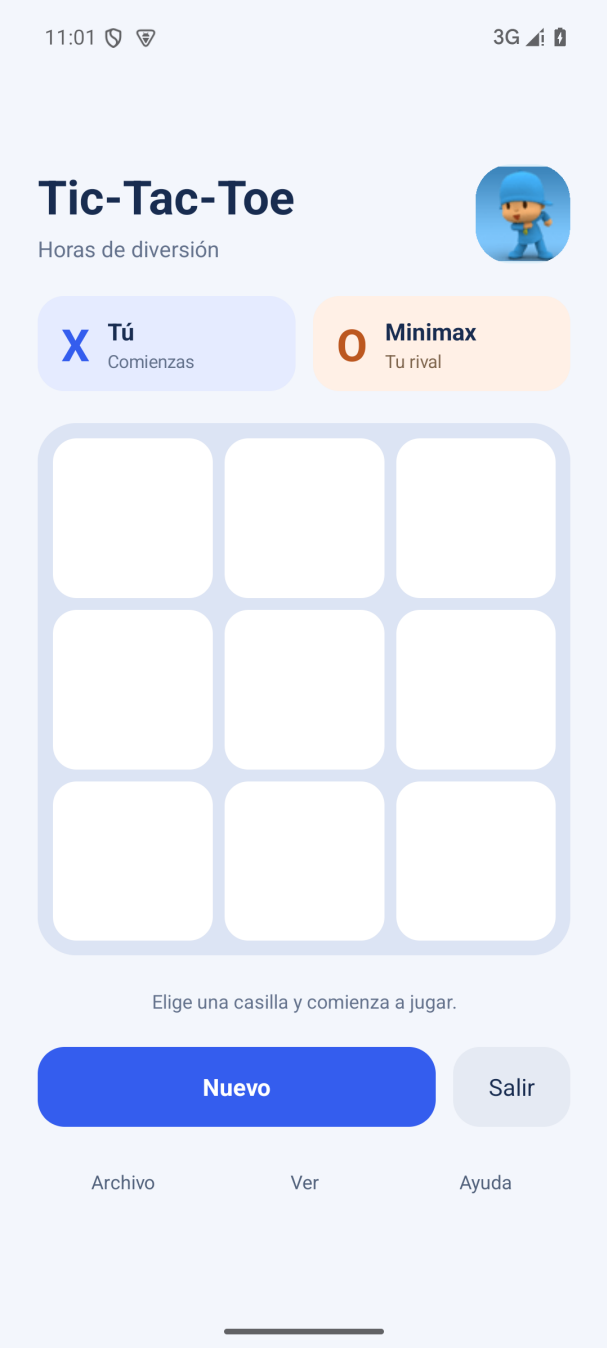


Figura A.14 Aplicación reabierta con una partida nueva y tablero vacío.